

COMP 4451: LLMs for Data Science and AI

Instructor: Dr. Sean Connin

Course GTA: Elijah Alabi (elijah.alabi@du.edu)

Assignment 1. Few-Shot Prompt Data Challenge

Overview

In this assignment, you will apply few-shot prompt engineering techniques to create effective prompts for data processing and cleaning using `health_dataset.csv`. You will then use these prompts with the OpenAI API and GPT-4o-mini to generate and evaluate language model outputs that can be compared to results produced programmatically (i.e., explicit Python code). The purpose of this exercise is to help you develop insight into the efficacy of these two approaches as relates to tabular data processing and cleaning.

Instructions for code completion are included in the file: `hw1_starter_code.py`

You may work with a partner on this homework.

Learning Objectives

By the end of this assignment, you will be able to:

1. Design effective prompts for data processing and cleaning
2. Identify patterns in prompt-response relationships
3. Create reusable prompt templates for common data science workflows
4. Communicate your findings in clear, well-structured, terms

Provided Materials

1. `health_dataset.csv`
2. `hw1_starter_code.py`
3. `hw1_assignment_description.docx`
4. `requirements.txt`

Setup Requirements

- Download the required files from the Canvas Course Site. See: Week 1: Assignment - Prompt Engineering for Data Processing

- Local virtual environment with the necessary packages installed: *pandas*, *numpy*, *openai*, *pathlib*, *dotenv*, *pathlib*. Please use the provided requirements.txt file to build your env.
- Access to OpenAI API (we will provide an API key the first day of class). Please feel free to use your own [OpenAI API key](#). After opening an account, you can access the [API platform](#) to create a key. The cost is nominal.

Homework Submission

Upload your homework to Canvas in a zip folder. The folder should include the following:

1. Three dataset files: `health_dataset.csv`, `prog_cleaned_health_data.csv`, `llm_cleaned_health_data.csv`. These will be exported automatically into local working directory, provided your completed script runs properly.
2. Your `output.log` file
3. Your completed `.py` script (`hw1_comparative_analysis.py`). Modify the title to include your first & last name. For example: `JaneDoe_hw1_comparative_analysis.py`
4. A word document with your responses to the homework questions below. Name this file “`hw1_firstlastname`”. If you work with a partner include both of your first and last names in the filename, separated by a “_”.

Questions (20 pts each)

Answer each question in your own words. Where a question asks you to cite specific results, reference the terminal output from your script (captured in `output.log`). Responses should be concise and well-structured; a strong answer to any question is typically one to two paragraphs.

1. Data Quality Audit and LLM Gap Analysis

Review `health_dataset.csv` and the STEP 1 section of your script output.

- a) List every category of data quality issue present in the dataset (missing values, outliers, invalid values, type inconsistencies, formatting inconsistencies, duplicates, cross-field contradictions, date/age violations, and MNAR patterns). For each category, give at least one specific example from the data (e.g., patient ID, column name, value).
- b) Using the STEP 2 and STEP 3 sections of your output, identify the specific issue categories where the LLM left a higher residual count than the programmatic cleaning. For each, explain what the LLM likely failed to do.

Note: LLMs do not see a table — they process CSV data as a flat sequence of text tokens. The model infers column alignment from position (the 4th value in each row maps to the 4th header), but this alignment is fragile and can break when rows vary in token length. Similarly, operations like date arithmetic or threshold comparisons

must be re-derived from token patterns on every row rather than executed as code. Use these characteristics to explain why the specific failures you observed are plausible.

2. **Missing Value Detection: Precision and Representation**

The programmatic pipeline detects 17 missing values across five variant types (NaN, string 'NA', string 'NULL', string 'missing', and empty string).

- a) Compare the programmatic missing-value count (from the DETAILED PER-DATASET ANALYSIS for ORIGINAL DATA) against what the LLM reported in its analysis. Identify which specific representations (e.g., 'NULL' in weight_kg, 'NA' in age) each method caught or missed and explain why the discrepancies occurred.
- b) After cleaning, the programmatic pipeline reports 5 remaining missing values. How many does the LLM-cleaned dataset have? Explain why the residuals differ and discuss what this implies for using LLM-based cleaning in a production data pipeline where auditability and reproducibility are required.

3. **Scalability: From 27 Rows to Production Scale**

The health dataset contains 27 rows and 13 columns. Consider how each cleaning method would behave as the dataset grows.

- a) Identify at least two specific programmatic detection functions from the starter code that you expect to work correctly at 100,000 rows without modification. Identify at least one that might produce different or degraded results at that scale and explain why.
- b) The LLM-based approach sends the entire dataset as text in a single API call. The gpt-3.5-turbo model has a context window of 16,385 tokens — this is the maximum combined size of the prompt (your instructions and examples), the dataset, and the model's response (the cleaned CSV). The health dataset's 27 rows use roughly 2,000 tokens; the prompt instructions and examples use roughly 1,000 tokens; and the model needs roughly 2,000–3,000 tokens for its response.

Using these numbers, estimate how many rows could fit in a single API call. What happens when a dataset exceeds this limit? Identify one additional practical concern (beyond the token limit) that would arise if you tried to work around this by splitting a large dataset into batches and cleaning each batch separately.

- c) Propose a hybrid architecture that uses the LLM for tasks it handles well and programmatic code for tasks where it does not. Reference specific issue categories from your STEP 2 output to justify the division of labor.

4. **Few-Shot vs. Zero-Shot: Experimental Design**

Your cleaning prompt in Section 4 uses few-shot examples before presenting the dataset to the LLM.

- a) Describe a concrete experimental procedure to test whether the few-shot examples improved results. Your procedure should specify: what you would change in the prompt, what metrics from the comparison report you would use to measure the difference, and how many runs you would need to account for LLM output variability.
- b) Suppose the zero-shot variant resolves the same number of issue categories as the few-shot variant, with total residual counts within 5% of each other across multiple runs. Provide two specific, distinct arguments for still preferring few-shot prompting in a production setting, and one argument for preferring zero-shot. Do not appeal to generic claims about quality; ground each argument in a concrete operational concern (e.g., output format reliability, token cost, maintenance burden). Search online for information that can help you address this question.